Tribhuvan University
Faculty of Humanities and Social Sciences

THRIFT STORE
An Online Second-Hand Goods Marketplace

A PROJECT REPORT

Submitted to
Department of Computer Application
NIST College

In partial fulfilment of the requirements for the Bachelors in Computer Application

Submitted by
Santosh Shrestha
(6-2-1014-26-2022)

4 April, 2026

Under the Supervision of
Samish Shrestha

Tribhuvan University

Faculty of Humanities and Social Sciences

NIST College

SUPERVISOR'S RECOMMENDATION

I hereby recommend that this project prepared under my supervision by Santosh Shrestha entitled **"THRIFT STORE": An Online Second-Hand Goods Marketplace"** in partial fulfillment of the requirements for the degree of Bachelor of Computer Application is recommended for the final evaluation.

.....

Signature

Samish Shrestha

SUPERVISOR

Department of Computer Application

NIST College, Banepa-09

Tribhuvan University

Faculty of Humanities and Social Sciences

NIST College, Banepa-09

LETTER OF APPROVAL

This is to certify that this project prepared by SANTOSH SHRESTHA entitled "**THRIFT STORE"** in partial fulfillment of the requirements for the degree of Bachelor in Computer Application has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required degree.

……. **SIGNATURE of Supervisor** Mr. Samish Shrestha Department of BCA NIST College, Banepa, Kavre	…….. **SIGNATURE of HOD/Coordinator** Mr. Samish Shrestha Department of BCA NIST College, Banepa, Kavre
……. **SIGNATURE of Internal Examiner** Internal Examiner	……. **SIGNATURE of External Examiner** External Examiner

ABSTRACT

Thrift Store is a full-stack web-based e-commerce platform developed to facilitate the buying and selling of second-hand goods in Nepal. The system is built using Node.js with Express.js framework on the backend, PostgreSQL as the relational database managed through Prisma ORM, and vanilla HTML/CSS/JavaScript with Tailwind CSS on the frontend. The platform integrates eSewa, Nepal's leading digital payment gateway, for secure online transactions using HMAC-SHA256 signature verification. Real-time chat functionality is implemented using Socket.io, enabling direct bidirectional communication between buyers and sellers.

To enhance platform intelligence and user experience, several algorithms are integrated. The Haversine Algorithm is used to calculate geographical distances between buyers and sellers for location-based filtering and delivery estimation. A dynamic ratings algorithm aggregates user feedback to compute seller credibility scores. Efficient product retrieval is achieved using optimized search algorithms with keyword matching and indexing. Additionally, a content-based filtering recommendation system suggests relevant products to users based on their browsing history and preferences.

The application follows a Model-View-Controller (MVC) architectural pattern with a service layer, ensuring clean separation of concerns. Key features include JWT-based user authentication, role-based access control, a product listing and admin approval workflow, granular order management with item-level status tracking, Firebase OTP-based phone verification, a seller rating system, and in-app push notifications. The platform is implemented in TypeScript for type safety and employs Prisma ORM for schema-driven database access. With 13 database models and over 30 REST API endpoints, Thrift Store represents a production-capable solution for Nepal's growing second-hand goods market.

Keywords: *E-Commerce, Thrift Store, Node.js, Express.js, TypeScript, PostgreSQL, Prisma, eSewa, Socket.io, JWT, Nepal*

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to my project supervisor, Samish Shrestha for their invaluable guidance, continuous support, and constructive suggestions throughout the development of this project. Their expertise and encouragement were instrumental in shaping this work.

I am also thankful to the Head of the Department and all the faculty members of the Department of Computer Application at NIST College for their support and motivation during the course of this project.

I extend my sincere thanks to Tribhuvan University and the Faculty of Humanities and Social Sciences for providing this opportunity to undertake such a practical and meaningful project.

Finally, I would like to thank my family and friends for their constant encouragement and support throughout the duration of this project.

Santosh Shrestha
6-2-1014-26-2022
BCA 6th Semester, NIST College

TABLE OF CONTENT

SUPERVISOR'S RECOMMENDATION ii
LETTER OF APPROVAL ii
ABSTRACT iii
ACKNOWLEDGEMENT iv
List of Tables vii
List of Figures viii
CHAPTER 1: INTRODUCTION 1
1.1 Introduction 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope and Limitation 2
1.5 Development Methodology 3
1.6 Report Organization 4
CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW 5
2.1 Background Study 5
2.1.1 E-Commerce and C2C Marketplaces 5
2.1.2 Node.js and Express.js 5
2.1.3 TypeScript 5
2.1.4 PostgreSQL and Prisma ORM 5
2.1.5 JWT Authentication and Bcrypt 6
2.1.6 Socket.io for Real-Time Communication 6
2.1.7 eSewa Payment Gateway 6
2.1.8 Firebase Phone Authentication and Multer 6
2.2 Literature Review 7
2.2.1 OLX Nepal and Facebook Marketplace (Local Context) 7
2.2.2 ThredUp and Poshmark (Global Context) 7
2.2.3 Modern Full-Stack Web Technologies 8
2.2.4 Trust and Reputation Systems in Online Marketplaces 8
2.2.5 Digital Payment Adoption and Security in Nepal 8
CHAPTER 3: SYSTEM ANALYSIS AND DESIGN 10
3.1 System Analysis 10
3.1.1 Requirement Analysis 10
3.1.2 Feasibility Analysis 11
3.1.3 Object Modelling: Object and Class Diagram 12
3.1.4 Dynamic Modelling: State and Sequence Diagram 12
3.2 System Design 13

3.2.1 Database Schema Design 13
3.2.2 Interface Design (UI/UX) 14
3.2.3 Sequence Diagram: eSewa Payment Flow 14
3.4 Algorithm Details 15
CHAPTER 4: IMPLEMENTATION AND TESTING 19
4.1 Implementation 19
4.1.1 Tools Used 19
4.1.2 Implementation Details of Modules 19
4.2 Testing 21
4.2.1 Unit Test Cases 22
4.2.2 System Test Cases 23
CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATIONS 26
5.1 Conclusion 26
5.2 Lesson Learnt / Outcome 26
5.3 Future Recommendations 27
References 28
APPENDIX 30

List of Tables

Table 3.1: Technology Stack Overview 20

Table 4.1: Tools Used 21

Table 4.2: Unit Test Cases 25

Table 4.3: System Test Cases 26

List of Figures

Figure 3.1: Use Case Diagram - Thrift Store 9

Figure 3.2: Object and Class Diagram 13

Figure 3.3: State Diagram – Order Lifecycle 14

Figure 3.4: Database Schema Design – Thrift Store 15

Figure 3.5: Sequence Diagram – eSewa Payment Flow 16

CHAPTER 1: INTRODUCTION

1.1 Introduction

In the contemporary digital landscape, Nepal's consumer economy is undergoing a significant transformation. The traditional method of buying and selling second-hand goods relies heavily on physical stores, word-of-mouth referrals, or general-purpose social media platforms that are not optimized for e-commerce. This unorganized market often leads to inefficiencies including lack of price transparency, absence of buyer-seller trust mechanisms, and no structured order management.

With the increasing penetration of internet services and digital payment platforms like eSewa and Khalti in Nepal, there is a viable opportunity to digitize the thrift goods market. The success of fintech solutions and ride-hailing services in Nepal demonstrates that Nepali consumers are increasingly comfortable transacting digitally. However, the second-hand goods segment has not benefited proportionally from this digital transformation.

Thrift Store is a full-stack web-based e-commerce platform developed to fill this gap. Built using Node.js with Express.js and TypeScript, PostgreSQL with Prisma ORM, and a Tailwind CSS frontend, the platform enables users to list pre-owned items for sale, browse and purchase products, communicate with sellers in real-time via Socket.io, and transact securely through eSewa. The platform also incorporates an admin approval workflow, phone OTP verification, a seller rating system, and in-app notifications, making it a comprehensive and production-capable solution for Nepal's thrift economy.

1.2 Problem Statement

Despite the growing interest in second-hand and sustainable shopping in Nepal, the current market structure remains fragmented. The specific problems identified include:

1. Lack of a Centralized Platform: There is no dedicated, trusted digital marketplace for second-hand goods in Nepal. Buyers and sellers rely on informal channels with no quality control.
2. Absence of Trust Mechanisms: Neither buyers nor sellers have verified records of each other. There is no rating or review system to build seller reputation.

3. No Digital Payment Integration: Transactions are mostly cash-based, which lacks security, transparency, and record-keeping. There is a clear need for integration of local payment gateways like eSewa.
4. Inefficient Communication: Buyers have no direct channel to communicate with sellers about product condition, negotiation, or delivery arrangements.
5. No Product Quality Control: Without a verification mechanism, low-quality or fraudulent listings diminish buyer trust and platform integrity.
6. No Order Tracking: Buyers have no visibility into the status of their purchases after payment, and sellers have no structured way to manage fulfillment.

Thrift Store addresses all of these issues by providing a structured, secure, and feature-rich digital marketplace.

1.3 Objectives

The general objective of this project is to develop a fully functional online platform for buying and selling second-hand goods in Nepal.

1. To implement a secure user authentication and authorization system with role-based access control (USER and ADMIN roles) using JWT and Bcrypt.
2. To integrate eSewa payment gateway using HMAC-SHA256 signature generation for secure digital transactions.
3. To implement a real-time bidirectional chat system using Socket.io for direct buyer-seller communication.
4. To develop a seller rating and review system (1-5 stars) linked to completed orders for trust building.
5. To implement phone OTP verification via Firebase to ensure user authenticity before product listing.
6. To build an admin panel for product approval management and platform oversight.

1.4 Scope and Limitation

1.4.1 Scope

The scope of Thrift Store includes the following functionalities:

1. User registration, login, and profile management with photo upload
2. Product listing with up to 5 images, title, description, price, category, gender, location, and refundability

3. Admin panel for product approval, rejection (with reason), and user management
4. Product browsing with category and seller filtering
5. Shopping cart and order placement with multi-product support
6. eSewa sandbox payment gateway integration
7. Real-time buyer-seller messaging using Socket.io
8. Order tracking with item-level status: Pending → Processing → Shipped → Delivered → Received
9. Seller rating (1-5 stars) and review system
10. Phone OTP verification via Firebase
11. In-app notification system for product and order status events

1.4.2 Limitations

12. The eSewa payment gateway is currently in sandbox (test) mode and has not been tested with live NPR transactions.
13. The platform is web-based only; native mobile applications for iOS/Android are not included.
14. Delivery logistics and courier integration are not implemented; physical delivery is arranged between buyer and seller.
15. Search functionality is basic (filter by category/seller); full-text search and AI recommendations are not included.
16. Real-time GPS tracking of sellers or delivery agents is not implemented.

1.5 Development Methodology

The development of Thrift Store follows an agile methodology adapted for an academic project timeline. Agile was selected for its flexibility, iterative delivery, and emphasis on continuous feedback allowing the project to evolve based on supervisor input and emerging technical requirements.

Instead of rigid, sequential phases, work was organized into time-boxed iterations (Sprints), each delivering a functional increment of the system. This approach enabled early validation of core features and reduced the risk of late-stage rework.

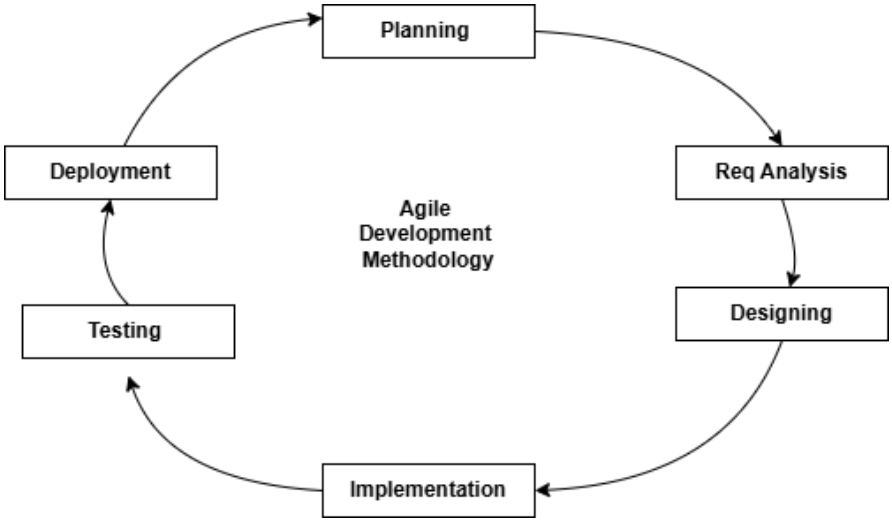


Figure 1.1 Agile Methodology

Agile Methodology was used in Thrift Store because it significantly benefited the development of Thrift Store by enabling adaptability, early validation, and effective risk management. As requirements evolved such as adding document uploads and camera-based verification Agile allowed smooth mid-project changes without disrupting progress. Core features were built and tested early, creating a stable base for advanced functionalities. High-risk components like payment integration and geolocation were handled in separate sprints, minimizing delays and ensuring focused development. This flexible approach resulted in a reliable, well-tested platform aligned with both academic and practical needs.

1.6 Report Organization

This report is organized into five main chapters. Chapter 1 provides the background, problem statement, objectives, scope, and methodology. Chapter 2 presents the background study covering all key technologies and a review of similar existing systems. Chapter 3 covers system analysis and design including use case descriptions, DFDs, ER diagram, architecture diagram, and data dictionaries. Chapter 4 details the implementation of all modules with tools used, and presents unit and system test cases. Chapter 5 concludes with lessons learned and future recommendations.

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

2.1.1 E-Commerce and C2C Marketplaces

Electronic commerce (e-commerce) refers to commercial transactions conducted over the internet. Thrift Store operates as a Consumer-to-Consumer (C2C) marketplace, a model where individual users can both list and purchase items from one another. Unlike B2C (Business-to-Consumer) platforms, C2C marketplaces require robust trust mechanisms such as ratings and verification systems, since transactions occur between private individuals without a corporate seller guarantee. The platform serves as a trusted intermediary providing verification, payment processing, and dispute awareness.

2.1.2 Node.js and Express.js

Node.js is a JavaScript runtime built on Chrome's V8 engine, designed for building scalable network applications using an event-driven, non-blocking I/O model. Express.js is a minimal and flexible Node.js web application framework that provides a robust set of features for building REST APIs. Thrift Store uses Express.js 5.2.1 as the backend framework to define route handlers, apply middleware, and serve static frontend files. The non-blocking nature of Node.js is particularly beneficial for real-time features like Socket.io chat.

2.1.3 TypeScript

TypeScript is a strongly-typed superset of JavaScript developed by Microsoft. It adds optional static typing, interfaces, and generics to JavaScript. The entire backend of Thrift Store is written in TypeScript 5.9.3, which significantly reduces runtime errors through compile-time type checking, improves code documentation, and enhances maintainability. TypeScript's integration with Prisma provides end-to-end type safety from database queries to API responses.

2.1.4 PostgreSQL and Prisma ORM

PostgreSQL is a powerful, open-source object-relational database system known for its reliability, extensibility, and ACID compliance. Prisma is a next-generation ORM for Node.js and TypeScript that provides a type-safe database client generated from the schema definition. Thrift Store uses Prisma 6.4.1 for all database interactions, with schema

migrations tracked in the prisma/migrations/ directory. This combination ensures type-safe queries, automatic schema evolution, and a visual database browser via Prisma Studio.

2.1.5 JWT Authentication and Bcrypt

JSON Web Tokens (JWT) are an open standard (RFC 7519) for securely transmitting information between parties as a signed JSON object. In Thrift Store, JWTs are generated upon login, containing the user's id, email, role, and name, with a 2-day expiration. Bcrypt is a password-hashing function that incorporates a salt to protect against rainbow table attacks. All user passwords are hashed using Bcrypt with a salt factor of 10 before storage, ensuring that even if the database is compromised, plaintext passwords cannot be recovered.

2.1.6 Socket.io for Real-Time Communication

Socket.io is a JavaScript library that enables real-time, bidirectional, event-based communication between web clients and servers. It builds on the WebSocket protocol but provides additional fallback mechanisms. Thrift Store uses Socket.io 4.8.3 to power its chat system. The Socket.io server applies JWT authentication middleware at the connection level, assigns users to personal notification rooms (user-{userId}) and conversation-specific rooms (chat-{conversationId}), and persists all messages to the database before broadcasting.

2.1.7 eSewa Payment Gateway

eSewa is Nepal's first and largest digital wallet and payment gateway, operated by F1Soft International Pvt. Ltd. It supports online payment integration through its RC-Epay API. Thrift Store integrates eSewa using HMAC-SHA256 digital signatures for payment request verification. The integration supports sandbox testing with merchant code EPAYTEST and handles payment callbacks to update transaction and order status in the database.

2.1.8 Firebase Phone Authentication and Multer

Firebase is a Google-backed mobile and web development platform. Thrift Store uses Firebase's phone authentication to generate custom tokens for OTP-based phone verification, required before users can list products. Multer is a Node.js middleware for handling multipart/form-data, primarily used for file uploads. The platform uses Multer to handle product image uploads (up to 5 files, 5MB each) and user profile photos, stored in dedicated directories with uniquely generated filenames.

2.2 Literature Review

The emergence of digital marketplaces has significantly transformed consumer-to-consumer (C2C) commerce, particularly in the domain of second-hand goods. Various platforms support such transactions; however, their effectiveness varies in terms of integration, trust mechanisms, and localization. This section reviews existing systems and supporting technologies to identify research gaps addressed by the proposed *Thrift Store* system.

2.2.1 OLX Nepal and Facebook Marketplace (Local Context)

OLX Nepal and Facebook Marketplace are widely used platforms for second-hand trading in Nepal. These platforms provide basic functionalities such as product listing and browsing, enabling direct interaction between buyers and sellers.

However, these systems primarily function as classified advertisement platforms rather than fully integrated e-commerce solutions. They lack built-in digital payment systems, requiring users to rely on external methods such as cash transactions or bank transfers [11]. Furthermore, there is no centralized administrative mechanism for verifying product authenticity or ensuring quality control, which increases the risk of fraudulent activities. Communication between users is typically conducted through external channels, reducing transparency and traceability of transactions. Additionally, the absence of order management and tracking features limits accountability and overall user experience. These limitations highlight the need for a more structured and secure marketplace system.

2.2.2 ThredUp and Poshmark (Global Context)

ThredUp and Poshmark are leading global platforms for second-hand e-commerce, offering advanced features that enhance usability and trust.
Poshmark integrates social commerce functionalities such as user interaction, product sharing, and seller rating systems, which improve engagement and trust among users. ThredUp, on the other hand, employs a curated product acceptance process to ensure quality and reduce information asymmetry in transactions. These design principles align with established best practices in modern e-commerce systems [1], [2].
Despite these advantages, both platforms are designed for Western markets and rely on infrastructure that is not fully compatible with the Nepali context. In particular, they do

not support local digital payment systems such as eSewa, and their logistics models assume highly developed delivery networks. Therefore, while they provide valuable design insights, their direct applicability in Nepal remains limited.

2.2.3 Modern Full-Stack Web Technologies

The development of scalable and efficient e-commerce platforms is supported by modern full-stack technologies. The PERN stack (PostgreSQL, Express.js, React, Node.js) provides a flexible and high-performance architecture for dynamic applications [4], [10]. The use of TypeScript enhances code reliability by enabling static type checking. Additionally, ORM tools such as Prisma simplify database operations through type-safe query generation, reducing common errors associated with raw SQL queries [3]. Real-time communication technologies such as Socket.IO further enhance user interaction by enabling instant messaging features [5]. These technologies collectively inform the architecture of the *Thrift Store* platform.

2.2.4 Trust and Reputation Systems in Online Marketplaces

Trust is a critical factor in online marketplaces. Research by Paul Resnick et al. demonstrates that reputation systems significantly improve transaction success rates and user confidence [8]. Their findings indicate that sellers with higher ratings are more likely to attract buyers and achieve better pricing outcomes.
To address this, the *Thrift Store* platform incorporates a transaction-based rating system (1–5 stars), ensuring that only verified purchases can generate reviews. Constraints are applied to prevent duplicate feedback, thereby maintaining the integrity of the system.

2.2.5 Digital Payment Adoption and Security in Nepal

Digital payment adoption in Nepal has increased significantly, with eSewa emerging as the dominant platform [11], [12]. Its widespread usage makes it an ideal choice for integration into local e-commerce systems.

Security is a critical aspect of digital transactions. Techniques such as Hash-based Message Authentication Code (HMAC) are widely used to ensure data integrity and prevent tampering [6]. Additionally, authentication mechanisms such as JSON Web Tokens (JWT) enhance system security by protecting user sessions [9]. These technologies are implemented in the *Thrift Store* platform to ensure secure and reliable transactions.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 System Analysis

3.1.1 Requirement Analysis

i. Functional Requirements

The functional requirements define what the system must do. These are illustrated via the Use Case Diagram in Figure 3.1. The main actors are: Guest User (browse only), Registered User / Buyer / Seller (full platform access), and Administrator (platform management).

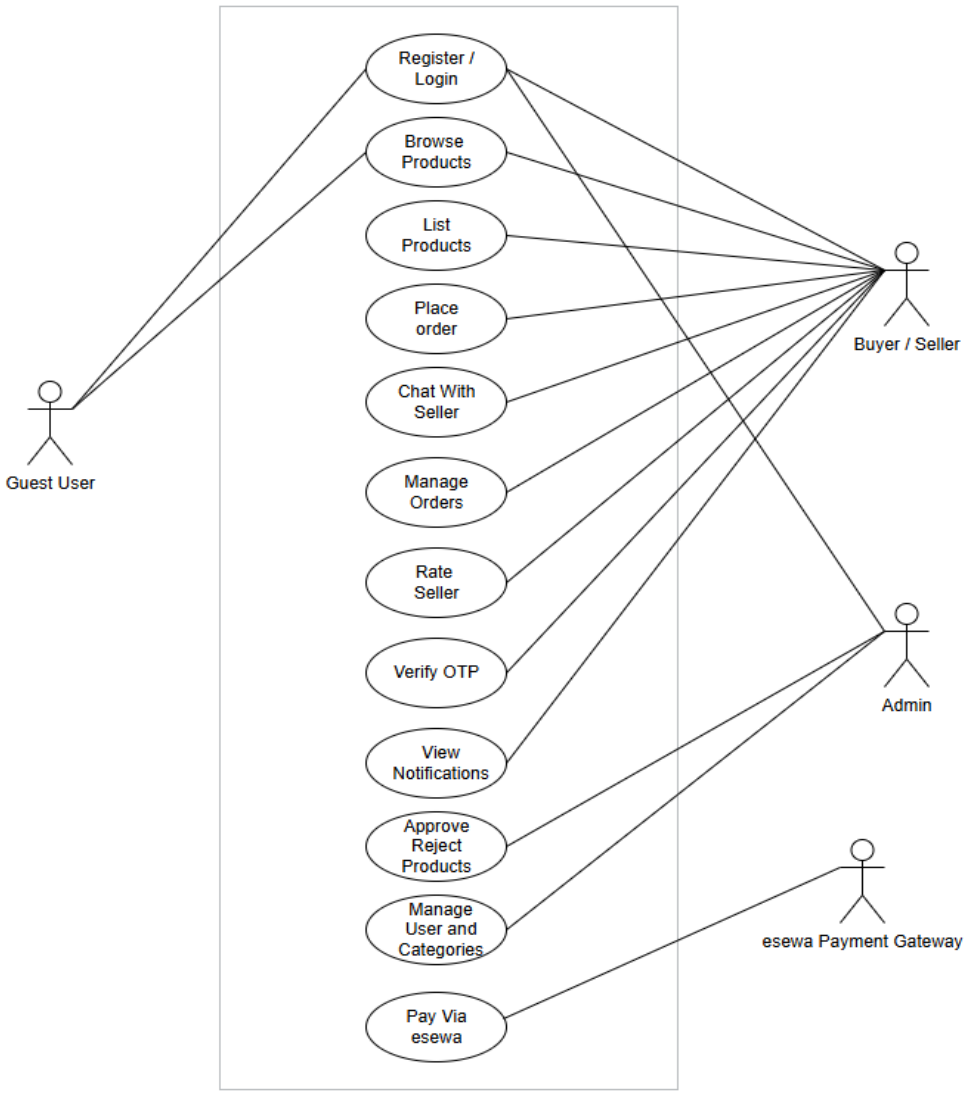


Figure 3.1: Use Case Diagram - Thrift Store

ii. Non-Functional Requirements

17. Availability: The system shall be available 99% of the time during operational hours.
18. Response Time: API endpoints shall respond within 2 seconds; real-time messages within 500ms.
19. Security: Passwords hashed with Bcrypt; API endpoints protected with JWT; payment requests signed with HMAC-SHA256.
20. Scalability: The MVC + service layer architecture and Prisma migrations allow horizontal and vertical scaling.
21. Maintainability: Full TypeScript codebase with strict mode, clean module boundaries, and centralized error handling.

3.1.2 Feasibility Analysis

Technical Feasibility

All technologies used (Node.js, Express.js, TypeScript, PostgreSQL, Prisma, Socket.io) are mature, open-source, and well-documented with large community support. The development machine (standard laptop with 8GB+ RAM) is sufficient for local development. eSewa provides free sandbox credentials. Firebase provides a free authentication tier. All tools are available at no cost, making the project technically feasible.

Operational Feasibility

The system is designed with an intuitive Tailwind CSS interface following standard web conventions, minimizing the learning curve for end users. The admin panel provides clear tools for product management. Buyer and seller workflows are clearly separated with role-based access control. The system can be operated by non-technical administrators once deployed.

Economic Feasibility

Development uses entirely open-source and free technologies. Cloud hosting for initial deployment can be done using free tiers of services such as Railway (PostgreSQL + Node.js), Vercel, or Render. The primary cost is development time. eSewa integration uses free sandbox credentials for testing. The economic cost of development and initial deployment is minimal.

3.1.3 Object Modelling: Object and Class Diagram

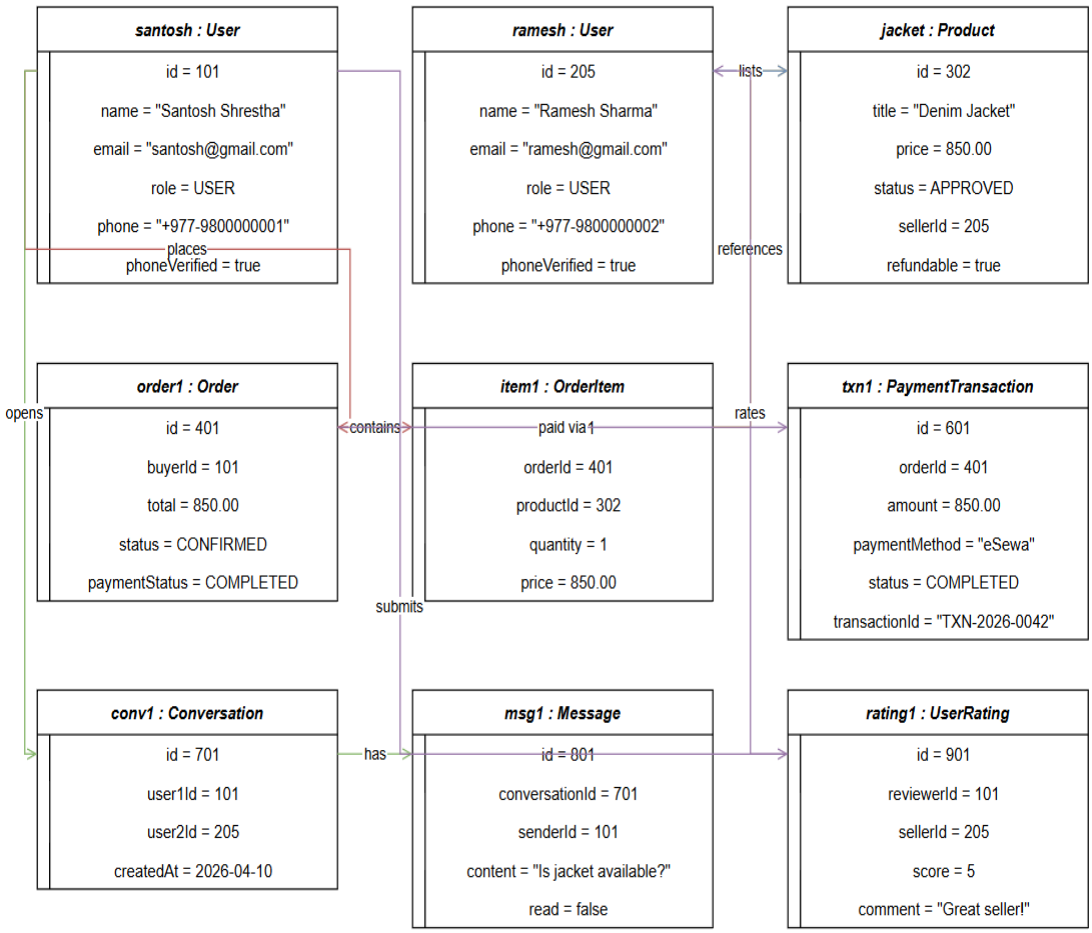


Figure 3.2: Object and Class Diagram

3.1.4 Dynamic Modelling: State and Sequence Diagram

State Diagram

A state diagram shows how an object transitions between different states in response to events.

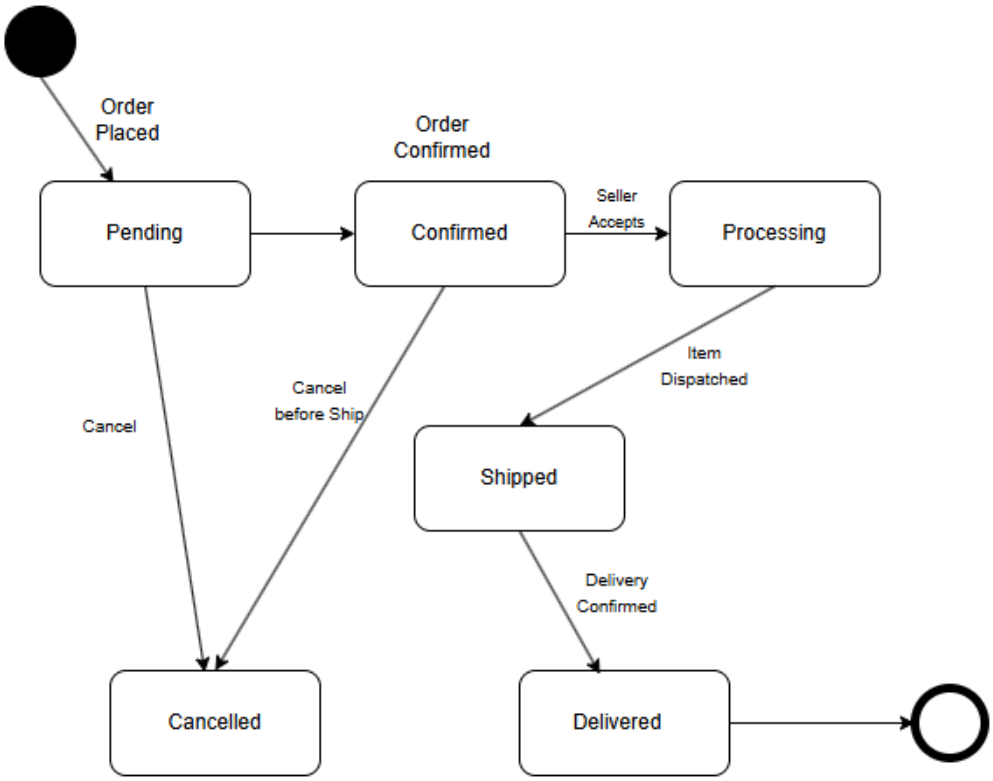


Figure 3.3: State Diagram – Order Lifecycle

3.2 System Design

3.2.1 Database Schema Design

The database is normalized to Third Normal Form (3NF) to reduce data redundancy and ensure referential integrity. All relationships are enforced via Foreign Keys in PostgreSQL. The schema is managed through Prisma migrations (13 migration files tracking schema evolution from initial setup through the chat system addition). The ER diagram below illustrates all entities and their relationships.

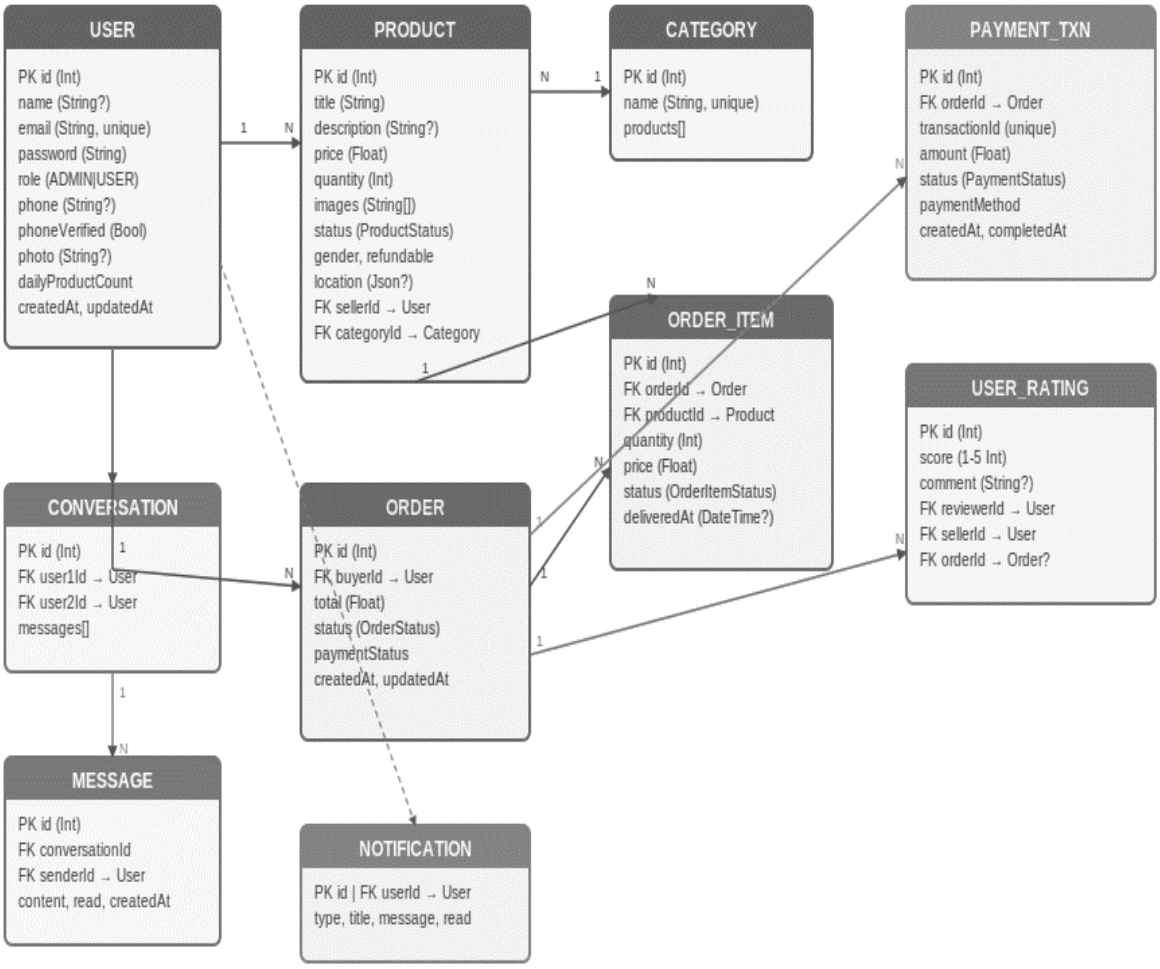


Figure 3.4: Database Schema Design- Thrift Store

3.2.2 Interface Design (UI/UX)

The frontend is designed as a multi-page application using HTML5 and Tailwind CSS for a consistent, responsive design system. Key interfaces include: index.html (landing page with hero section), all-items.html (product catalog with category filters), product-details.html (product view with seller info and purchase option), checkout.html (order summary and eSewa payment), orders.html (buyer order history), seller-orders.html (seller order management), chat-inbox.html (real-time messaging), admin.html (product approval panel), and seller-profile.html (public seller profile with ratings).

3.2.3 Sequence Diagram: eSewa Payment Flow

The sequence diagram below illustrates the complete payment flow from order placement through eSewa callback verification.

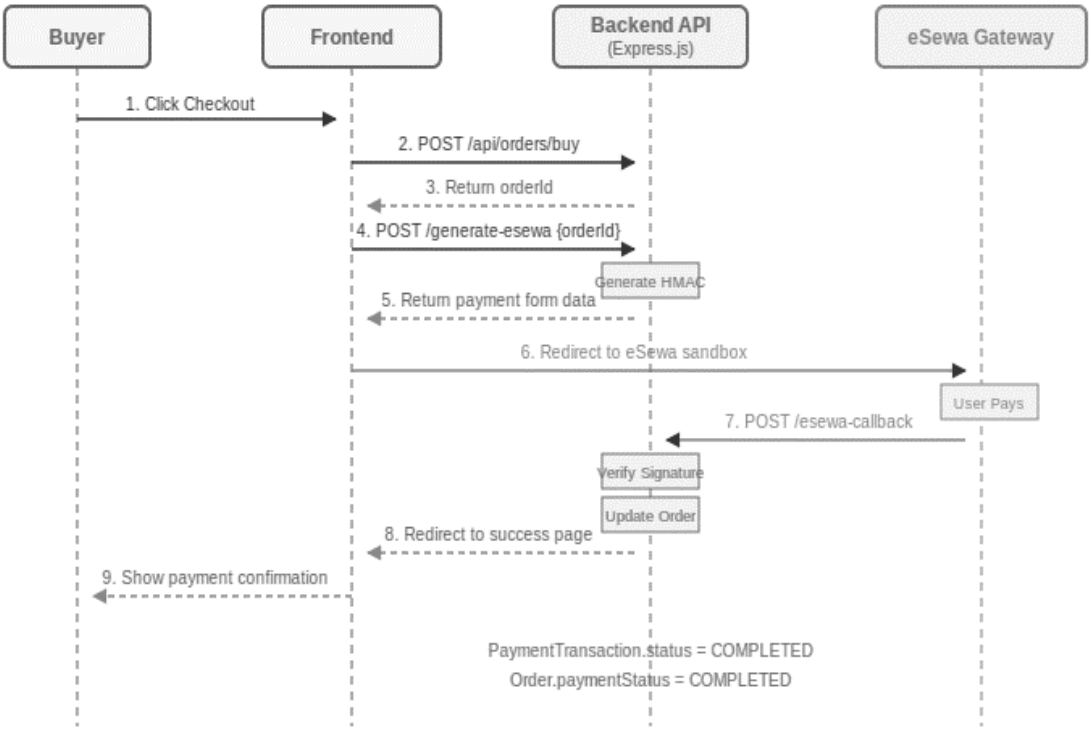


Figure 3.5: Sequence Diagram - eSewa Payment Flow

3.4 Algorithm Details

A key algorithmic component of the thrift store e-commerce system is the intelligent product discovery, recommendation, and location-based services mechanism.

Algorithm 3.4.1: Haversine Distance Calculation

Purpose: Calculate distance between two geographic coordinates.

Input

lat1, lon1 (Decimal): First location coordinates
lat2, lon2 (Decimal): Second location coordinates

Output

distance (Decimal): Distance in kilometers

Pseudocode

```
FUNCTION calculateHaversineDistance(lat1, lon1, lat2, lon2):
   R ← 6371
   φ1 ← lat1 × (π / 180)
   φ2 ← lat2 × (π / 180)
   Δφ ← (lat2 - lat1) × (π / 180)
   Δλ ← (lon2 - lon1) × (π / 180)
```

```
    a ← sin²(Δφ / 2) + cos(φ1) × cos(φ2) × sin²(Δλ / 2)
    c ← 2 × atan2(√a, √(1 - a))
    distance ← R × c

    RETURN distance
END FUNCTION
```

Algorithm 3.4.2: Product Search Algorithm

Purpose: Retrieve products based on user keyword input.

Input

query (String), product_list (List)

Output

matched_products (List)

Pseudocode

```
FUNCTION searchProducts(query, product_list):
    query ← toLowerCase(query)
    matched_products ← empty list

    FOR EACH product IN product_list DO
        IF query IN product.title OR query IN product.description THEN
            APPEND product TO matched_products
        END IF
    END FOR

    RETURN matched_products
END FUNCTION
```

Algorithm 3.4.3: Product Rating Algorithm

Purpose: Calculate average rating.

Input

ratings_list (List)

Output

average_rating (Decimal)

Pseudocode

```
FUNCTION calculateAverageRating(ratings_list):
    total ← 0
    count ← length(ratings_list)

    IF count = 0 THEN
        RETURN 0
    END IF
```

```
    FOR EACH rating IN ratings_list DO
        total ← total + rating
    END FOR

    RETURN total / count
END FUNCTION
```

Algorithm 3.4.4: Content-Based Filtering

Purpose: Recommend similar products.

Input

target_product, product_list

Output

recommended_products (List)

Pseudocode

```
FUNCTION recommendProducts(target_product, product_list):
    recommended_products ← empty list

    FOR EACH product IN product_list DO
        score ← 0

        IF product.category = target_product.category THEN
            score ← score + 1
        END IF

        IF product.brand = target_product.brand THEN
            score ← score + 1
        END IF

        IF product.color = target_product.color THEN
            score ← score + 1
        END IF

        IF product.price_range = target_product.price_range THEN
            score ← score + 1
        END IF

        IF score ≥ 2 THEN
            APPEND product TO recommended_products
        END IF
    END FOR

    RETURN recommended_products
END FUNCTION
```

Component	Technology	Version	Purpose
Runtime	Node.js	LTS	JavaScript server execution
Backend Framework	Express.js	5.2.1	REST API + static file serving
Language	TypeScript	5.9.3	Type-safe backend development
Database	PostgreSQL	15+	Relational data persistence
ORM	Prisma	6.4.1	Type-safe DB access + migrations
Authentication	jsonwebtoken	9.0.3	JWT token generation and verification
Password Hash	Bcrypt	6.0.0	Secure password storage
File Upload	Multer	2.1.1	Product & profile image handling
Real-time	Socket.io	4.8.3	WebSocket chat & notifications
Email	Nodemailer	8.0.1	Email delivery
HTTP Client	Axios	1.13.2	Server-to-server HTTP calls
Validation	class-validator	0.14.3	Input validation decorators
Payment	eSewa RC-Epay	Sandbox	Nepali digital payment gateway
Phone OTP	Firebase Admin SDK	Latest	Custom OTP token generation
Frontend CSS	Tailwind CSS	3.x	Utility-first responsive styling
CASE Tool	Draw.io / Prisma Studio	Latest	Diagrams + DB management
Version Control	Git & GitHub	Latest	Source code management
IDE	Visual Studio Code	Latest	Development environment

Table 3.1: Technology Stack Overview

Chapter 4: Implementation and Testing

4.1 Implementation

4.1.1 Tools Used

Category	Tool / Technology	Version	Purpose
Runtime	Node.js	LTS	JavaScript server runtime
Framework	Express.js	5.2.1	REST API framework
Language	TypeScript	5.9.3	Type-safe backend
Database	PostgreSQL	15+	Relational data storage
ORM	Prisma	6.4.1	Database access & migrations
Auth	jsonwebtoken + Bcrypt	9.0.3 / 6.0.0	JWT + password hashing
File Upload	Multer	2.1.1	Multipart image uploads
Real-time	Socket.io	4.8.3	WebSocket messaging
Payment	eSewa RC-Epay	Sandbox	Digital payment processing
Phone Auth	Firebase Admin SDK	Latest	OTP verification
Validation	class-validator	0.14.3	Input validation
Frontend	Tailwind CSS + Vanilla JS	3.x / ES6+	Responsive UI
IDE	Visual Studio Code	Latest	Development environment
Version Control	Git & GitHub	Latest	Source code management
DB Browser	Prisma Studio	Built-in	Visual database management
Diagram Tool	Draw.io	Latest	UML and flow diagrams

Table 4.1: Tools Used

4.1.2 Implementation Details of Modules

A) Authentication Module (src/auth/)

The authentication module handles user registration and login. During registration, the password is hashed using bcrypt.hash() with salt factor 10. The photo upload is handled by Multer, saving to /public/uploads/users/ with a unique filename. Login verifies the submitted password against the stored hash using bcrypt.compare(). On success, a JWT is

generated with payload {id, email, role, name} and 2-day expiration. The auth middleware (auth.middleware.ts) extracts the Bearer token from the Authorization header, verifies it using the JWT_SECRET_KEY, and attaches the decoded payload to req.user for use by downstream handlers.

B) Product Management Module (src/product/)

Product creation passes through a middleware pipeline: (1) auth.middleware — verifies JWT; (2) phone verification check — queries User.phoneVerified; (3) productLimit.middleware — checks dailyProductCount against limit; (4) upload.middleware — processes up to 5 image files via Multer, validating MIME type (JPEG/PNG/GIF/WebP) and size (max 5MB each). The product is saved with PENDING status. The admin endpoint GET /api/products/admin/pending returns all PENDING products. PATCH operations update status to APPROVED or REJECTED with a rejectionReason. On approval, a Notification record is created and emitted to the seller's Socket.io room.

C) Order Management Module (src/order/)

When a buyer POSTs to /api/orders/buy with [{productId, quantity}], the service validates each product's availability and APPROVED status, calculates the total, creates one Order record and N OrderItem records within a Prisma transaction to ensure atomicity. The order status machine transitions: PENDING → PROCESSING (when seller processes first item) → PARTIALLY_DELIVERED (when some items are delivered) → DELIVERED (all items delivered) → CANCELLED. Item status has its own six-state machine: PENDING → PROCESSING → SHIPPED → DELIVERED → RECEIVED → CANCELLED. Sellers update items via PUT /api/orders/item/:orderItemId/status.

D) eSewa Payment Module (src/esewa/)

The payment flow: (1) POST /api/payment/simple/generate-esewa receives orderId, retrieves order total, generates transactionId (TT-{timestamp}-{6-char-random}), calculates breakdown (subtotal, tax 13%, service charge NPR 30, delivery NPR 100), generates HMAC-SHA256 signature, creates PENDING PaymentTransaction, returns payment form data. (2) Frontend renders hidden form and auto-submits to eSewa sandbox URL. (3) POST /api/payment/simple/esewa-callback receives eSewa's signed response, verifies HMAC signature, updates PaymentTransaction.status to COMPLETED/FAILED, and updates Order.paymentStatus accordingly.

E) Real-Time Chat Module (src/chat/)

The Socket.io server applies authentication middleware on every connection: the JWT from socket.handshake.auth.token is verified, and userId is extracted and stored on the socket. On connection, the socket automatically joins the personal room user-{userId}. The joinChat event handler adds the socket to room chat-{conversationId} after verifying the user is a participant. The sendMessage handler validates content, persists the Message record to the database via Prisma, then emits the message event to all sockets in the conversation room. This ensures delivery to all active participants and persistence for offline users.

F) Seller Rating Module (src/ratings/)

POST /api/ratings/rate-seller validates the score (1-5 integer) and checks the unique constraint on (reviewerId, sellerId, orderId) before creating the UserRating record. The uniqueness is enforced both by a Prisma unique constraint at the schema level and a try-catch in the service layer. GET /api/ratings/seller/:sellerId is a public endpoint that returns all ratings for a seller along with aggregated average score, providing potential buyers with reputation information.

G) Notification System

The Notification model stores all platform events with typed NotificationType enums: PRODUCT_SUBMITTED, PRODUCT_APPROVED, PRODUCT_REJECTED, ORDER_PLACED, ORDER_STATUS_CHANGED, ITEM_STATUS_CHANGED, DAILY_LIMIT_REACHED. Each notification has a title, message, read flag, and optional JSON metadata. Real-time delivery is achieved via Socket.io: after creating the database record, the server emits a notification event to the user's personal room (user-{userId}). Unread notifications can be retrieved via GET /api/notifications and marked read via PATCH.

4.2 Testing

Testing was conducted across two phases: unit testing of individual API endpoints and system testing of end-to-end workflows. All tests were performed manually using Postman for API testing and browser-based testing for frontend flows.

4.2.1 Unit Test Cases

TC No.	Test Case	Input Data	Expected Output	Actual Output	Status
TC -01	User Registration - Valid	name: 'Ram Thapa', email: 'ram@test.com', password: 'Test@1234'	HTTP 201; user created; JWT token returned	HTTP 201; account created; token returned	PASS
TC -02	User Registration - Duplicate Email	email: 'ram@test.com' (existing)	HTTP 400; 'Email already exists'	HTTP 400; correct error message	PASS
TC -03	Login - Valid Credentials	email: 'ram@test.com', password: 'Test@1234'	HTTP 200; JWT token with {id, email, role}	HTTP 200; valid JWT returned	PASS
TC -04	Login - Wrong Password	email: 'ram@test.com', password: 'WrongPass'	HTTP 401; 'Invalid credentials'	HTTP 401; correct error message	PASS
TC -05	Create Product - Valid (Phone Verified)	Auth token + product data + 2 images	HTTP 201; product created with PENDING status	HTTP 201; product in PENDING status	PASS
TC -06	Create Product - Phone Not Verified	Auth token; phoneVerified=false	HTTP 403; 'Phone not verified'	HTTP 403; correct error message	PASS
TC -07	Create Product - Daily Limit Exceeded	Auth token; dailyCount at limit	HTTP 429; DAILY_LIMIT_REACHED notification	HTTP 429; notification created	PASS
TC -08	Admin Product Approval	Admin JWT + productId	HTTP 200; product status = APPROVED	HTTP 200; status updated; notification sent	PASS
TC -09	Place Order - Valid	Auth token + [{productId:1, quantity:1}]	HTTP 201; Order + OrderItems created	HTTP 201; order and items created	PASS
TC -10	Place Order - Out of Stock	productId with quantity=0	HTTP 400; 'Product unavailable'	HTTP 400; correct error	PASS

TC -11	Generate eSewa Payment	Auth token + valid orderId	HTTP 200; payment form data with HMAC sig	HTTP 200; valid signature returned	PASS
TC -12	Rate Seller - Valid	Auth token + score:4, comment:'Good'	HTTP 201; UserRating created	HTTP 201; rating saved	PASS
TC -13	Rate Seller - Duplicate	Same reviewer + seller + orderId	HTTP 409; 'Already rated'	HTTP 409; unique constraint message	PASS
TC -14	Unauthenticated API Access	GET /api/orders/my-orders (no token)	HTTP 401; 'No token provided'	HTTP 401; correct rejection	PASS

Table 4.2: Unit Test Cases

4.2.2 System Test Cases

TC No.	Test Scenario	Steps	Expected Result	Status
ST-01	Complete Registration and Login Flow	1. Open register page 2. Fill all fields with valid data 3. Submit registration 4. Navigate to login 5. Enter credentials and submit	User registered successfully; JWT stored in localStorage; user redirected to home	PASS
ST-02	Product Listing and Admin Approval Flow	1. Login as seller 2. Verify phone via OTP 3. Fill product form with 3 images 4. Submit listing 5. Login as admin 6. Approve the product	Product visible on public all-items page after admin approval; seller receives PRODUCT_APPROVED notification	PASS
ST-03	Complete eSewa Purchase Flow	1. Login as buyer 2. Browse and select product 3. Place order 4. Click Pay via eSewa 5. Complete sandbox payment	Order status updated to PROCESSING; paymentStatus = COMPLETED; buyer receives ORDER_PLACED notification	PASS

ST-04	Seller Order Management Flow	1. Login as seller 2. View seller-orders page 3. Update item status to Processing 4. Update to Shipped 5. Update to Delivered	Each status update persists correctly; buyer sees updated item status; ITEM_STATUS_CHANGED notification delivered	PASS
ST-05	Real-Time Chat Flow	1. Login as buyer 2. Open product detail page 3. Click 'Chat with Seller' 4. Send a message 5. Login as seller in another tab 6. Observe message delivery	Message delivered to seller in real-time without page refresh; message persisted in database	PASS
ST-06	Phone Verification Flow	1. Login as unverified user 2. Attempt to list product (expect 403) 3. Navigate to phone verification 4. Enter phone number 5. Enter OTP and verify	403 error before verification; after OTP verified, product listing succeeds; User.phoneVerified = true	PASS
ST-07	Product Rejection and Notification	1. Seller creates product 2. Admin navigates to pending products 3. Admin rejects with reason 'Low quality images' 4. Seller checks notifications	Rejection notification with rejection reason delivered to seller; product status = REJECTED	PASS
ST-08	Seller Rating Flow	1. Buyer receives order items 2. Navigate to seller-profile page 3. Submit rating: 5 stars, 'Excellent item' 4. Refresh seller profile	Rating saved; seller's average rating displayed on profile; second rating attempt for same order blocked	PASS

Table 4.3: System Test Cases

All 14 unit test cases and 8 system test cases executed successfully, confirming that the core functional requirements of the Thrift Store platform — authentication, product lifecycle, order management, eSewa payment processing, real-time chat, and ratings — are implemented correctly and behave in accordance with their specifications.

CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATIONS

5.1 Conclusion

Thrift Store is a fully functional, feature-rich online marketplace for second-hand goods, developed as the capstone project for the Bachelor of Computer Application program at Tribhuvan University. The project successfully achieves all stated objectives, delivering a comprehensive e-commerce platform tailored for the Nepali market with features that address real gaps in the current second-hand goods ecosystem.

The platform demonstrates the practical application of modern full-stack web development including Node.js, Express.js, TypeScript, PostgreSQL, Prisma ORM, Socket.io, eSewa payment gateway integration, and Firebase OTP authentication. Complex features such as real-time bidirectional messaging, HMAC-SHA256 cryptographic payment signatures, a multi-stage order management state machine, and a phone-verification-gated product listing system have been successfully implemented and tested.

The development process reinforced the value of TypeScript for large-scale backend development, Prisma ORM for schema-driven database management, and the MVC+service layer pattern for maintainable code organization. The incremental development methodology allowed for progressive feature integration while maintaining system stability. Overall, Thrift Store represents a production-capable solution that can meaningfully contribute to Nepal's growing digital commerce landscape.

5.2 Lesson Learnt / Outcome

1. TypeScript's static type system significantly reduces runtime errors in complex backend systems. The investment in type definitions pays off during refactoring and debugging. Prisma's generated types create end-to-end type safety from database to API response.
2. Integrating a real payment gateway requires meticulous handling of cryptographic signatures, edge cases in callback processing, and thorough sandbox testing. The HMAC-SHA256 implementation provided hands-on experience in applied cryptography.

3. Socket.io's room-based architecture maps naturally to chat applications. Authenticating sockets at the connection level (not just per-message) is essential for security in real-time systems.
4. Prisma migrations enable iterative schema evolution without data loss. Planning the database schema carefully from the beginning (including status enumerations) saved significant refactoring effort compared to schema-less approaches.
5. Centralized error handling middleware in Express.js with a consistent JSON response format ({message, status, errors}) greatly simplifies API debugging and improves the developer experience for frontend integration.
6. The MVC+service layer pattern proved valuable for code organization in a complex application with 9 modules, making it easier to add features (like the chat and rating systems) without disrupting existing functionality.
7. Working with Nepal-specific technologies (eSewa, Firebase with Nepali phone numbers) provided practical experience in localizing e-commerce solutions for emerging markets.

5.3 Future Recommendations

1. Mobile Application: Develop native iOS and Android applications using React Native to increase platform accessibility and reach Nepal's predominantly mobile internet users.
2. AI-Based Recommendations: Integrate a recommendation engine that suggests products based on browsing history, purchase patterns, and similar user behavior.
3. Live Payment Integration: Transition the eSewa integration from sandbox to production mode and add support for Khalti, ConnectIPS, and mobile banking for broader payment coverage.
4. Real-Time GPS Tracking: Implement real-time location tracking for delivery using WebSockets to allow buyers to track their orders in transit.
5. Dispute Resolution: Add a formal dispute resolution system with admin arbitration for handling buyer-seller conflicts over product condition or delivery issues.
6. Multi-language Support: Add Nepali (Devanagari) language localization to make the platform accessible to users with limited English proficiency.

References

[1] M. C. Christensen and R. H. Thayer, The Project Manager's Guide to Software Engineering's Best Practices. IEEE Computer Society Press, 2001.

[2] A. H. Hofmann, Scientific Writing and Communication: Papers, Proposals, and Presentations, 3rd ed. Oxford University Press, Nov. 2016.

[3] Prisma, "Prisma ORM Documentation," prisma.io. [Online]. Available: https://www.prisma.io/docs. [Accessed: 2026].

[4] Express.js, "Express 5.x - Node.js web application framework," expressjs.com. [Online]. Available: https://expressjs.com. [Accessed: 2026].

[5] Socket.io, "Socket.IO Documentation," socket.io. [Online]. Available: https://socket.io/docs/v4. [Accessed: 2026].

[6] eSewa Pvt. Ltd., "eSewa RC-Epay Developer Documentation," developer.esewa.com.np. [Online]. Available: https://developer.esewa.com.np. [Accessed: 2026].

[7] Firebase, "Firebase Authentication - Phone Number," Google. [Online]. Available: https://firebase.google.com/docs/auth/web/phone-auth. [Accessed: 2026].

[8] P. Resnick, R. Zeckhauser, E. Friedman, and K. Kuwabara, "Reputation Systems," Communications of the ACM, vol. 43, no. 12, pp. 45–48, Dec. 2000.

[9] JWT.io, "JSON Web Tokens - Introduction," Auth0. [Online]. Available: https://jwt.io/introduction. [Accessed: 2026].

[10] PostgreSQL Global Development Group, "PostgreSQL 15 Documentation," postgresql.org. [Online]. Available: https://www.postgresql.org/docs/15/. [Accessed: 2026].

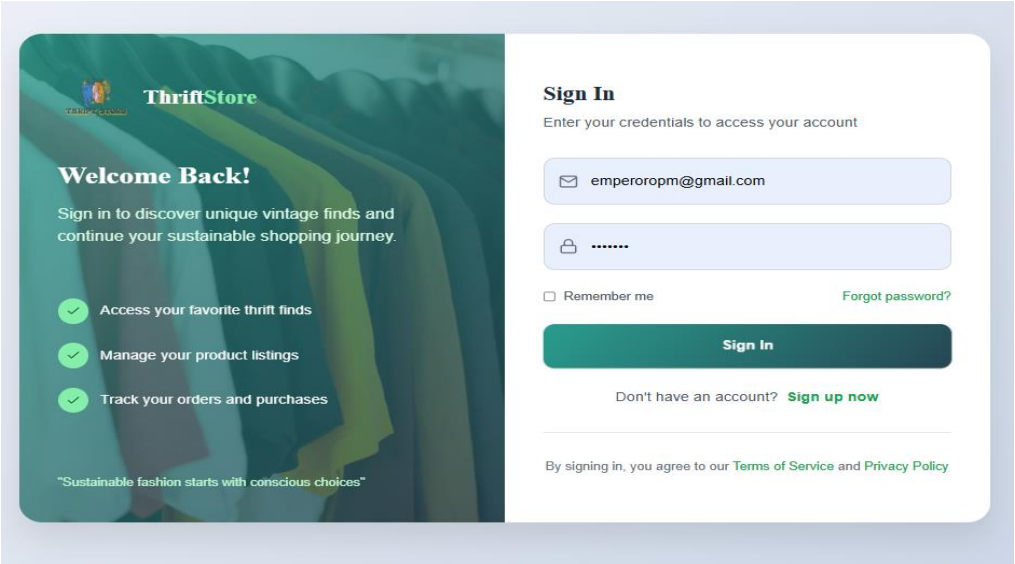
[11] Nepal Telecommunications Authority, "MIS Annual Report 2022/23," NTA, Kathmandu, Nepal, 2023. [Online]. Available: https://nta.gov.np. [Accessed: 2026].

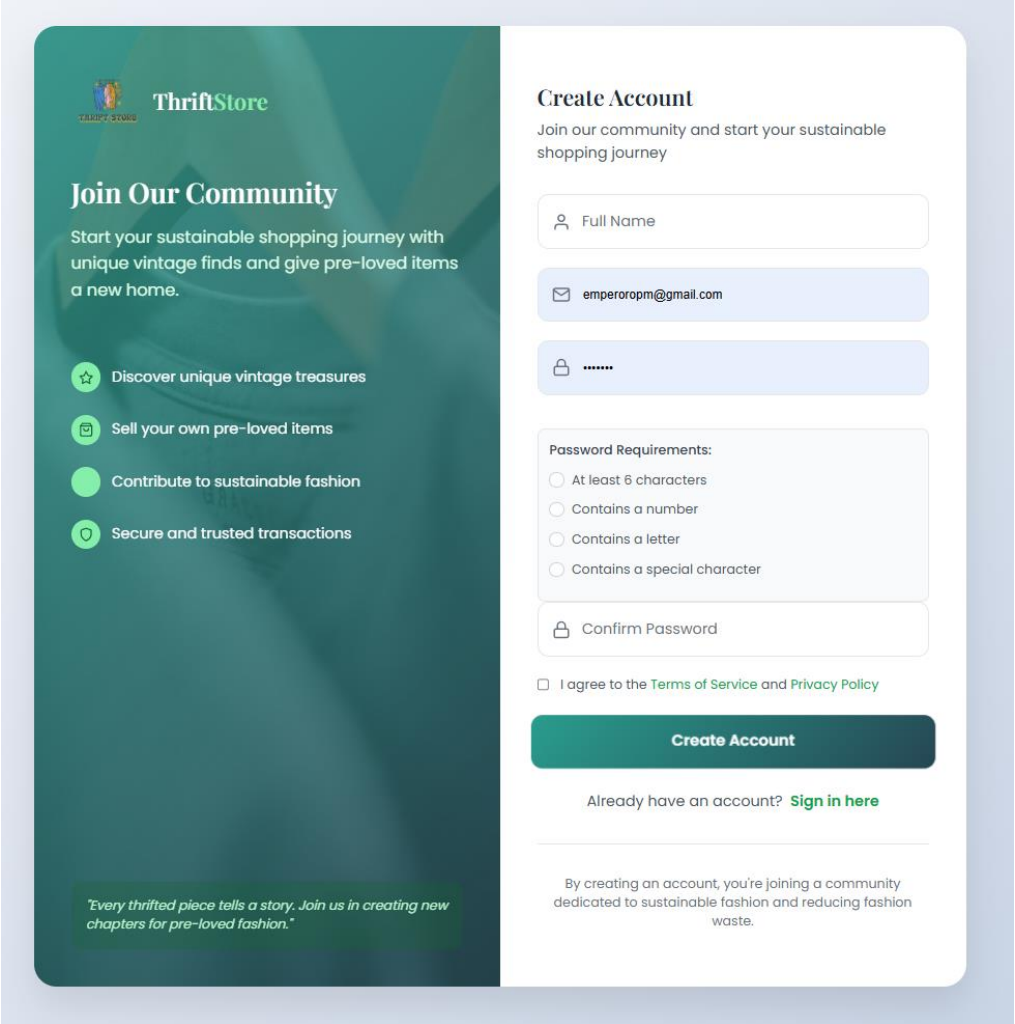
[12] F1Soft International Pvt. Ltd., "eSewa Digital Wallet," esewa.com.np. [Online]. Available: https://esewa.com.np. [Accessed: 2026].

[13] Multer Contributors, "Multer - Node.js middleware for multipart/form-data," GitHub. [Online]. Available: https://github.com/expressjs/multer. [Accessed: 2026].

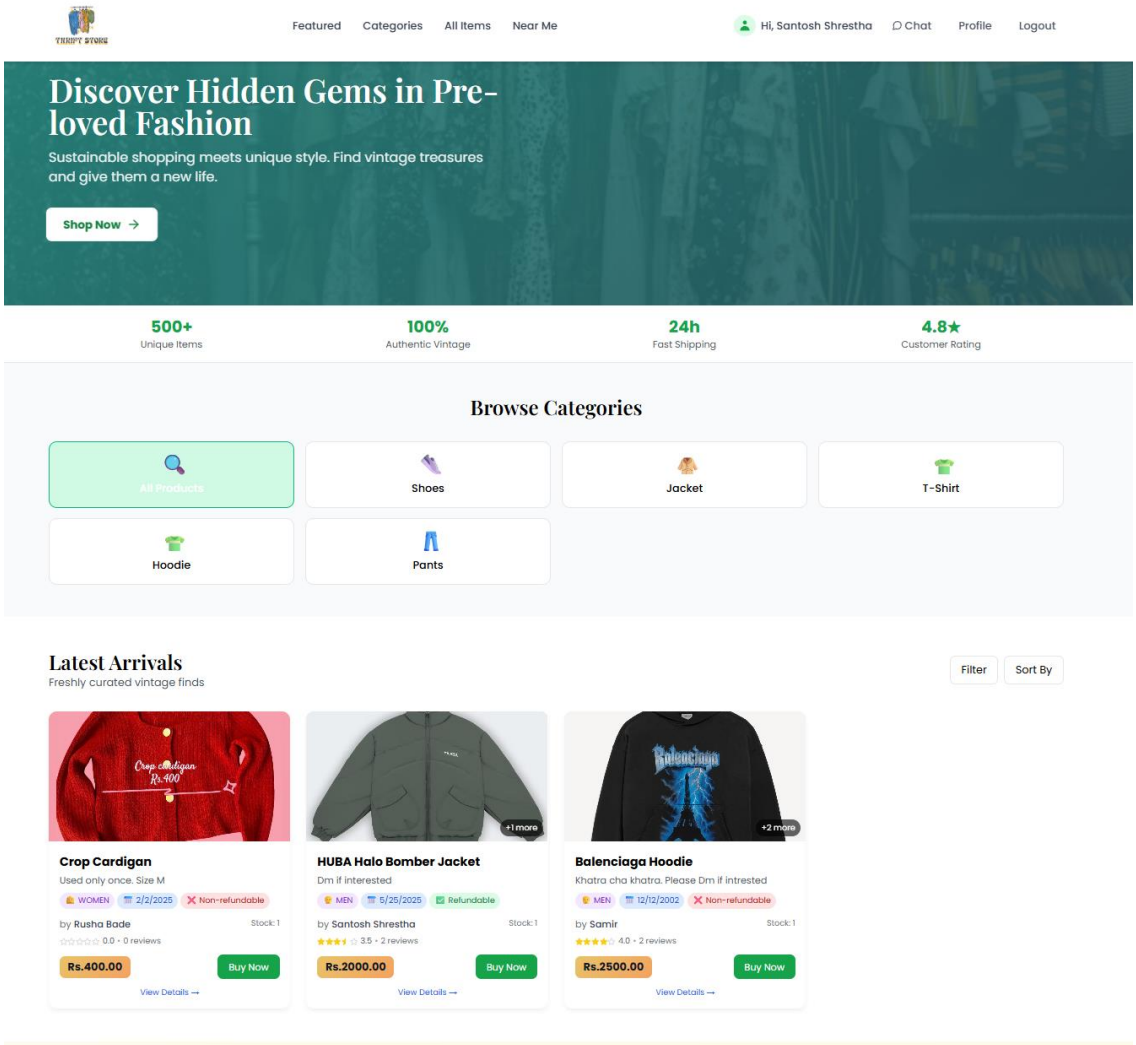
APPENDIX

1. Login and Signup Page

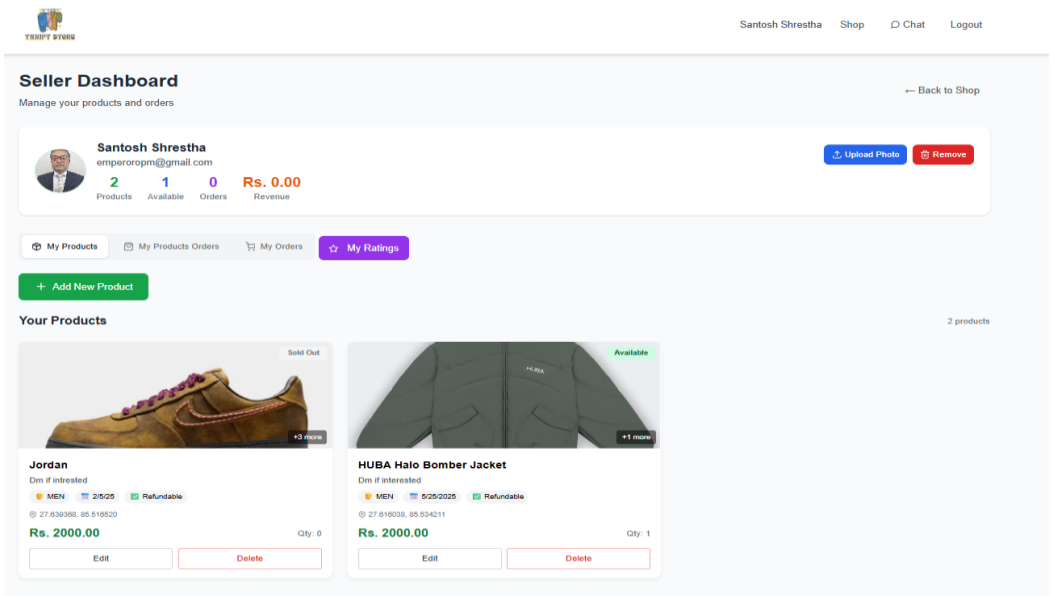




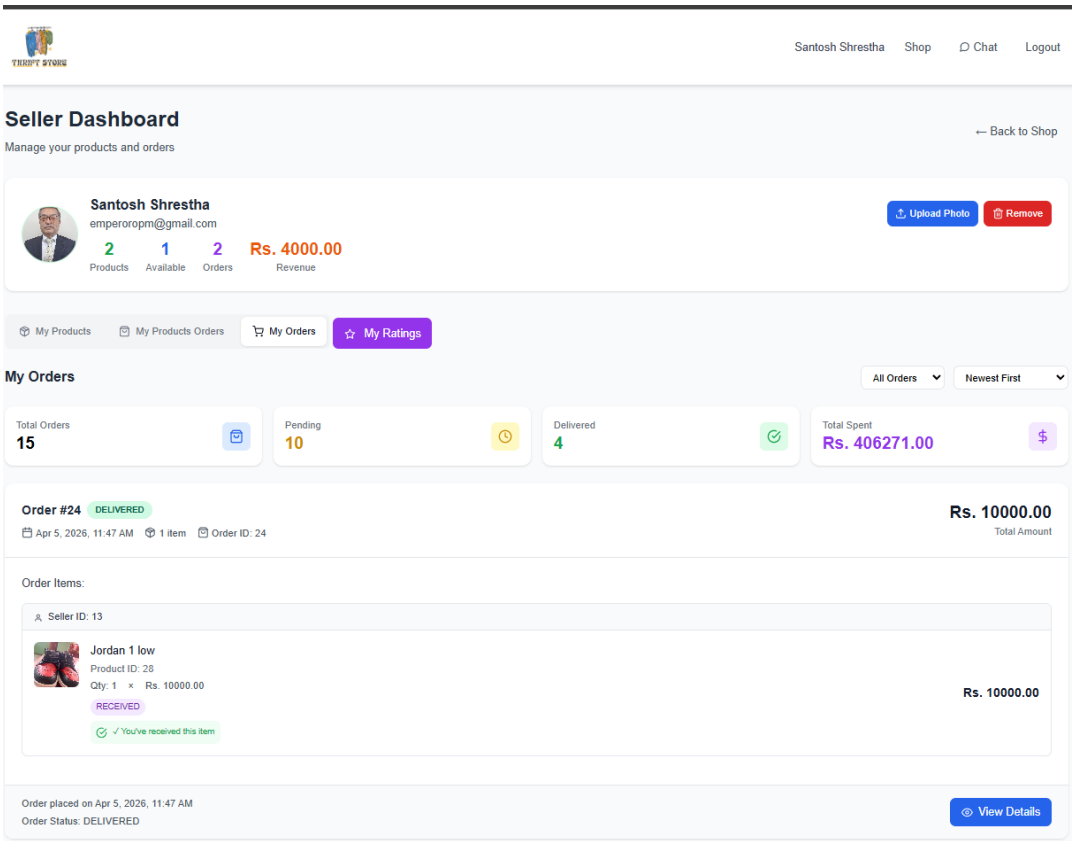
2. Home Page



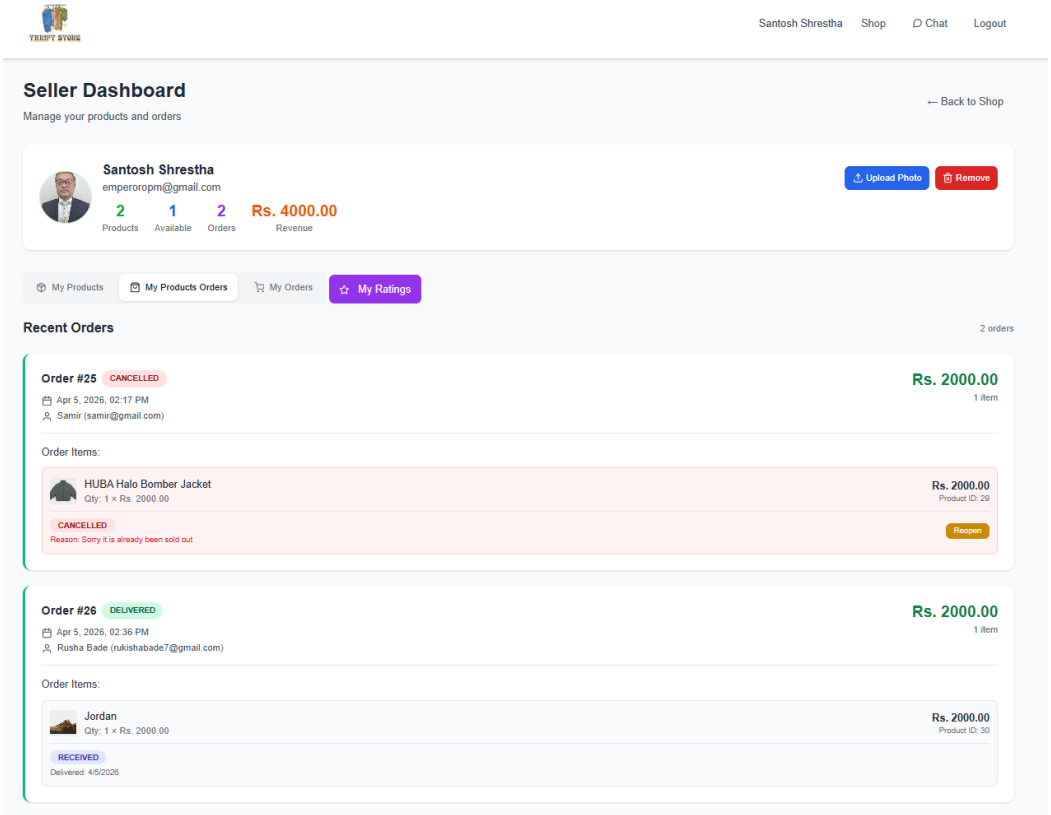
3. Profile



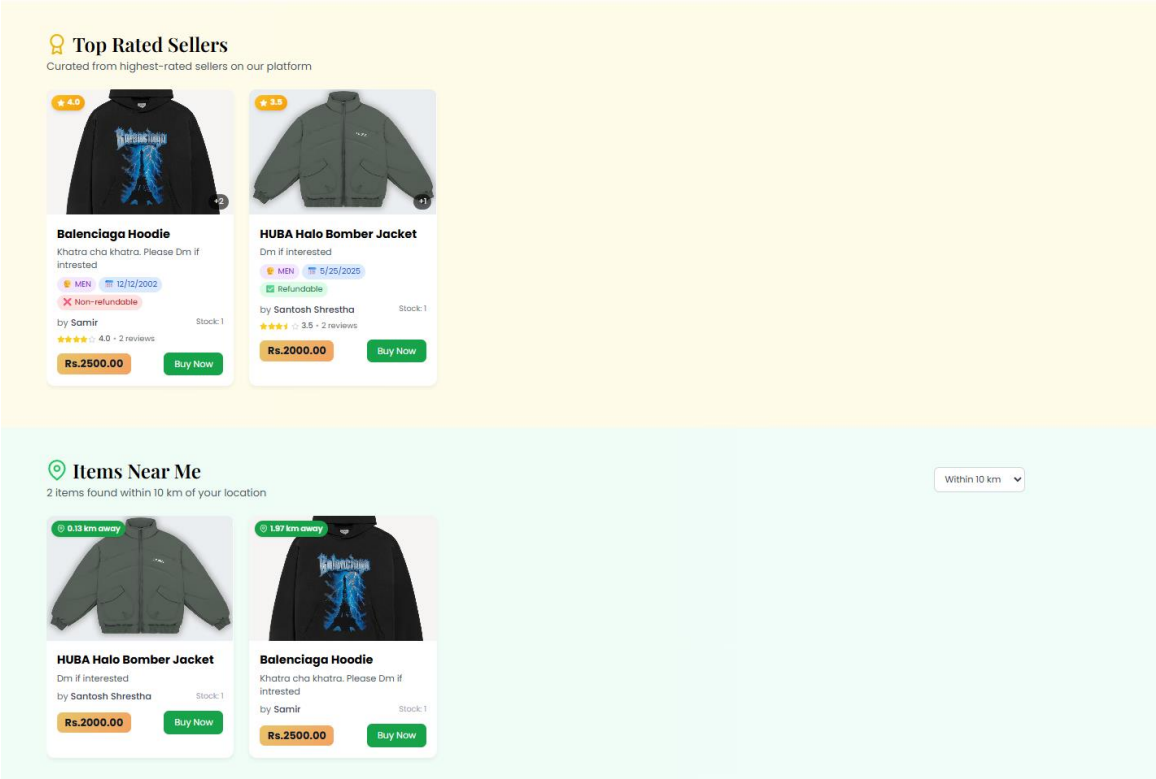
4. Order Page of Buyer



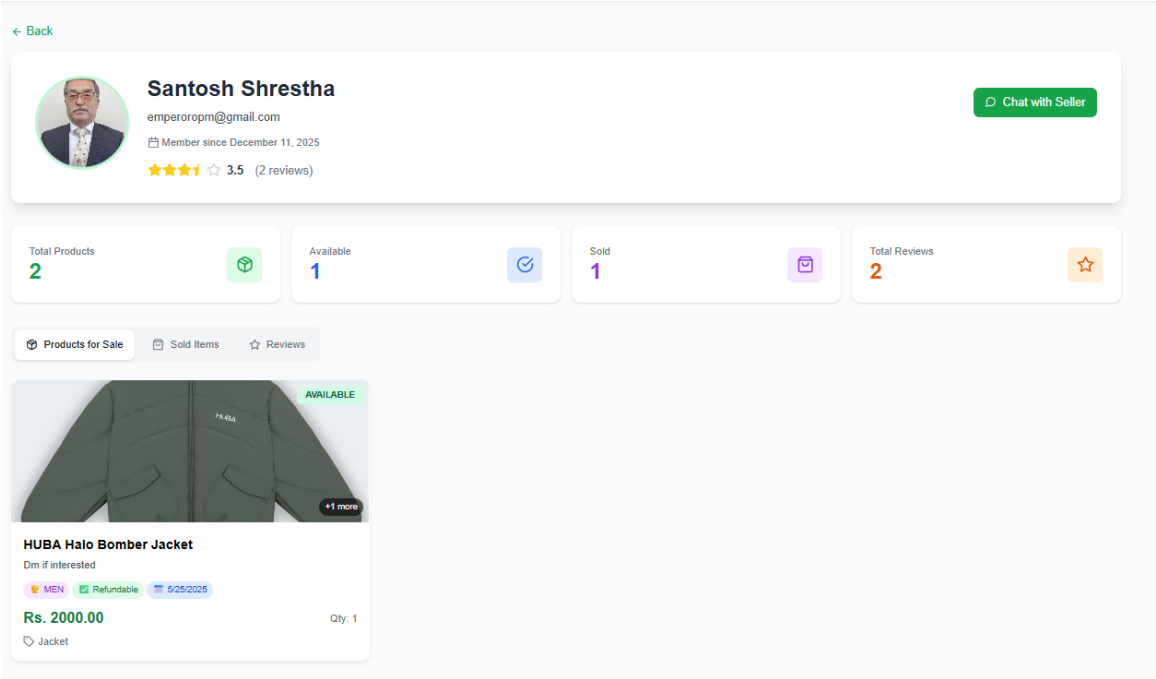
5. Order Page of Seller



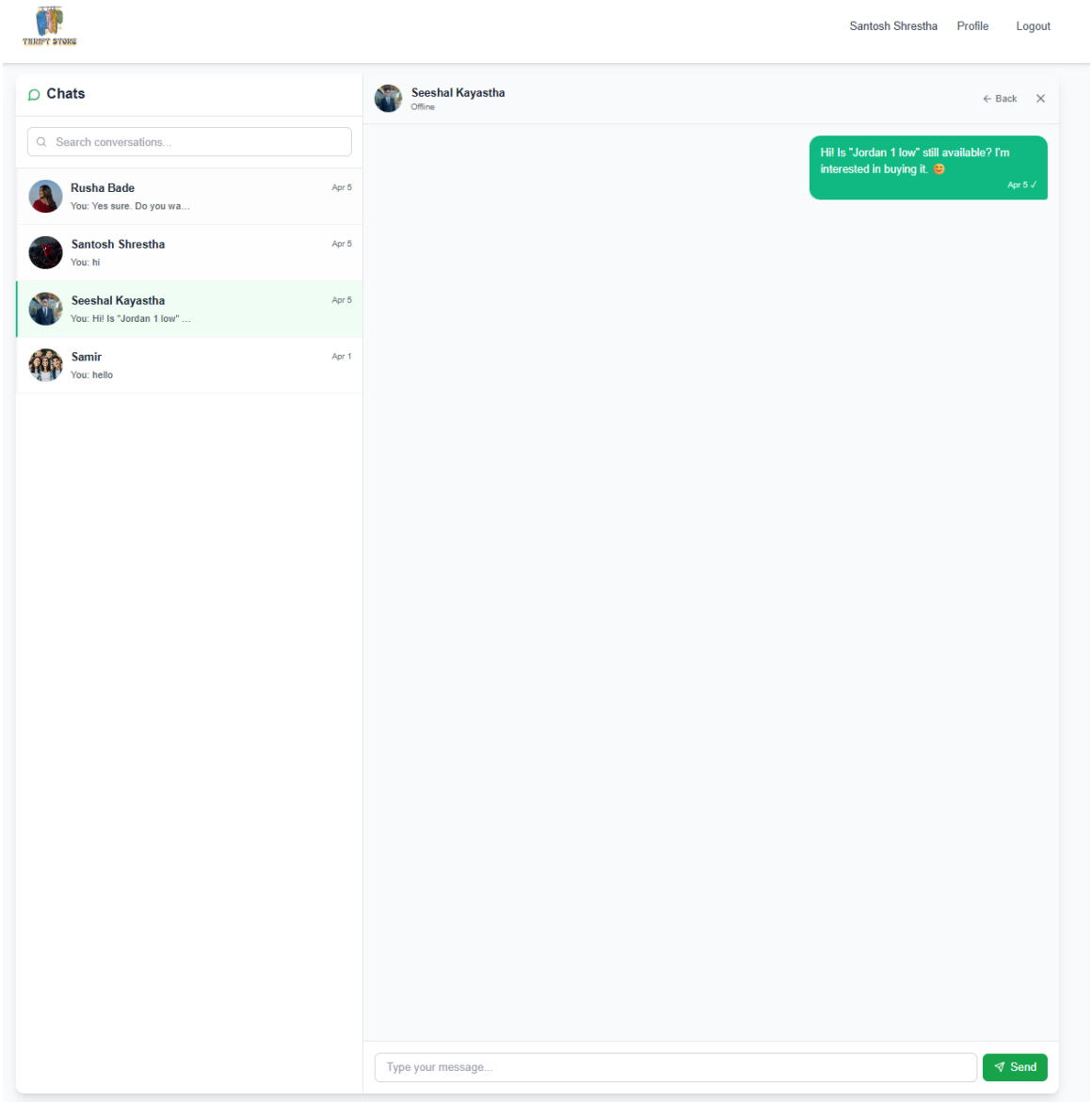
6. Top Rated and Items Nearby



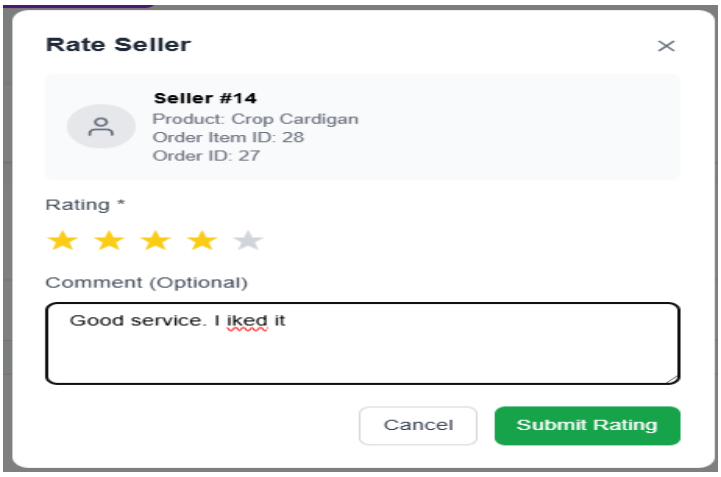
7. Seller Information



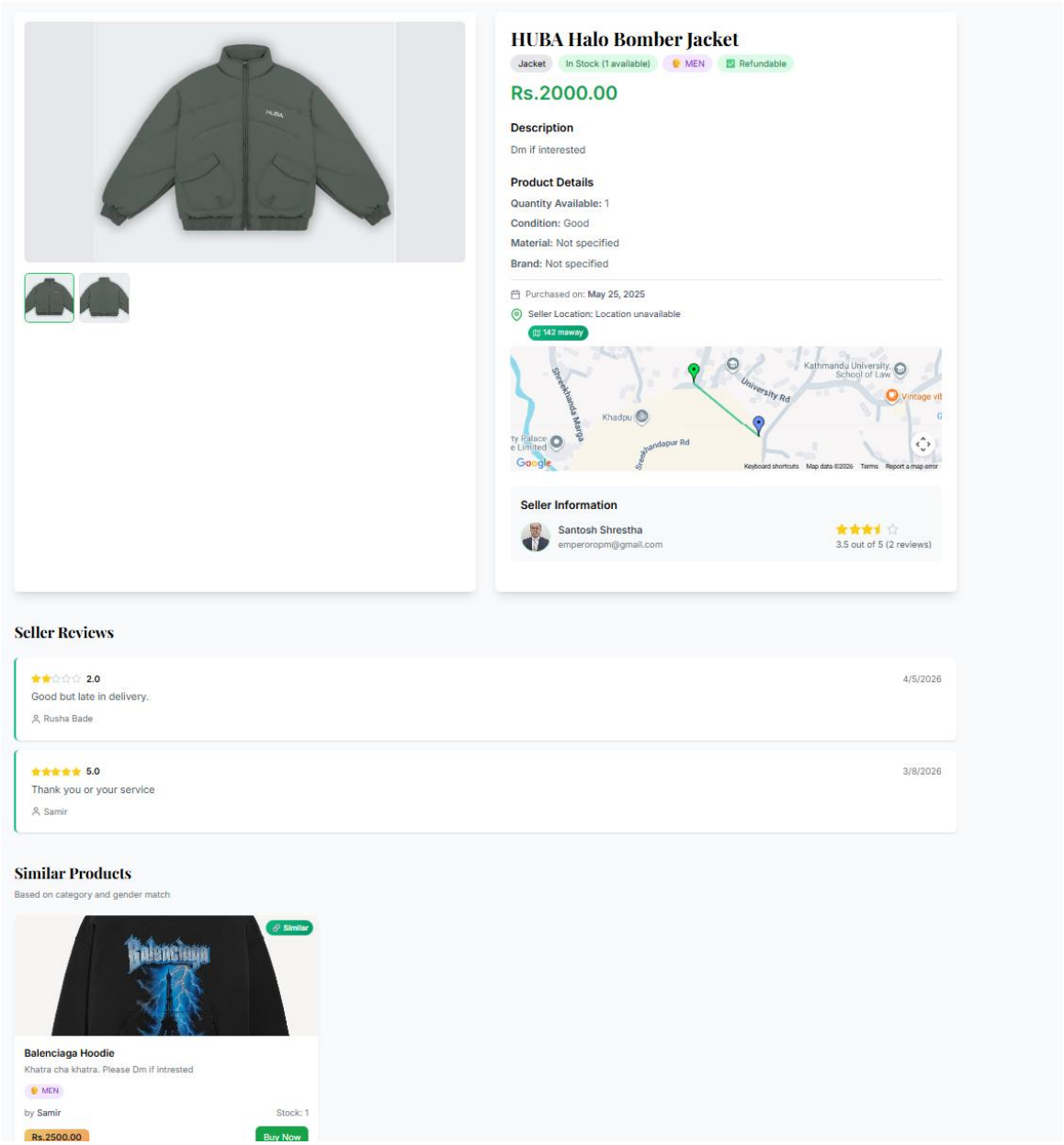
8. Chat Page



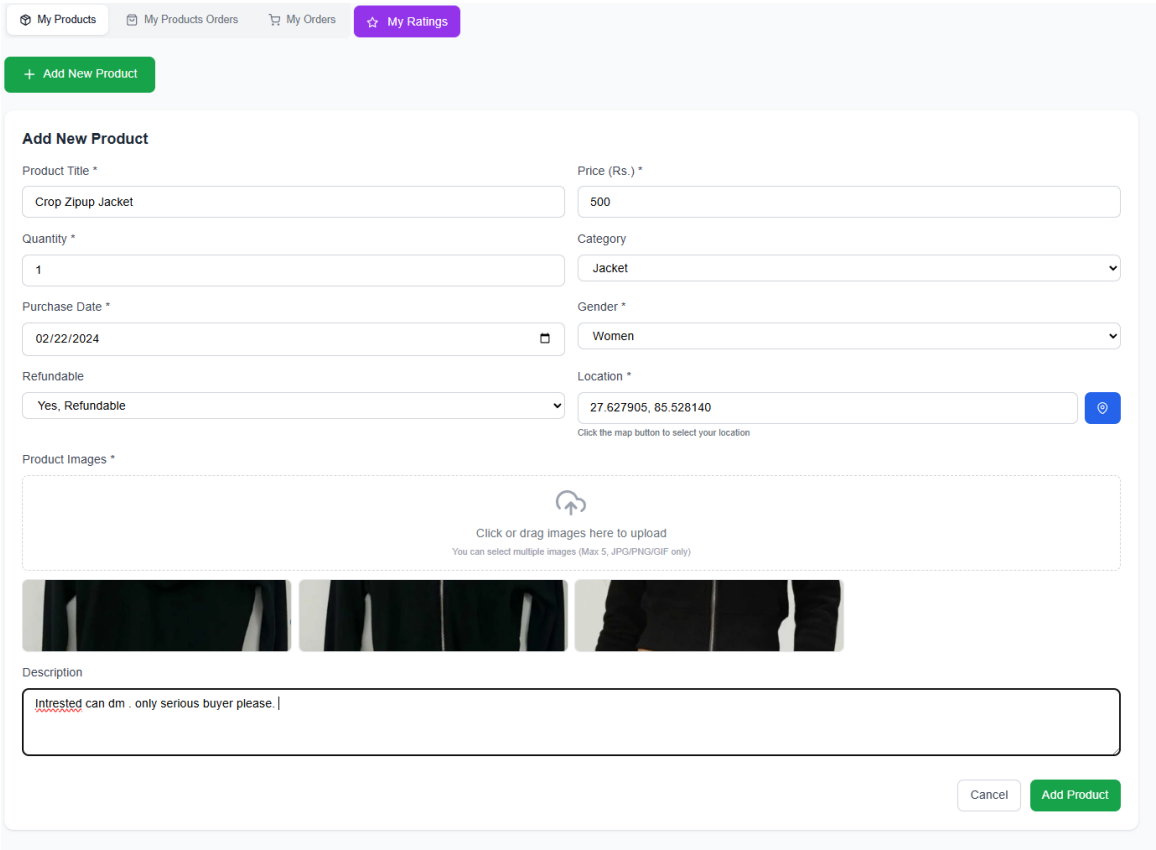
9. Rate Seller page



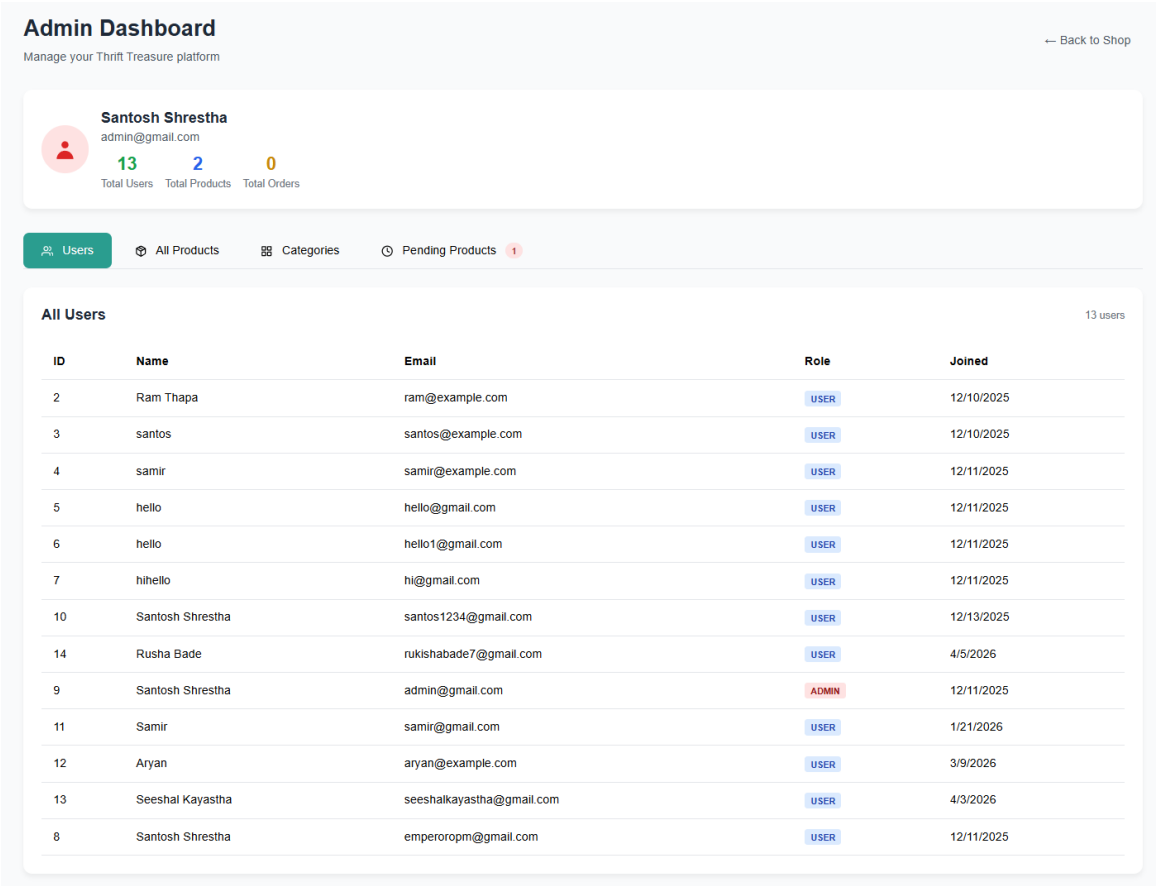
10. Product Details



11. Add Products Section

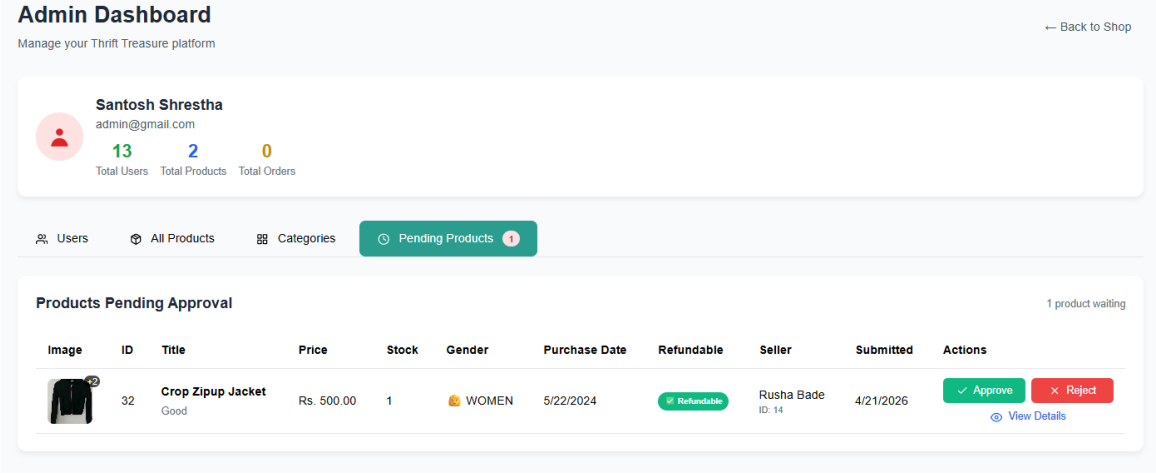
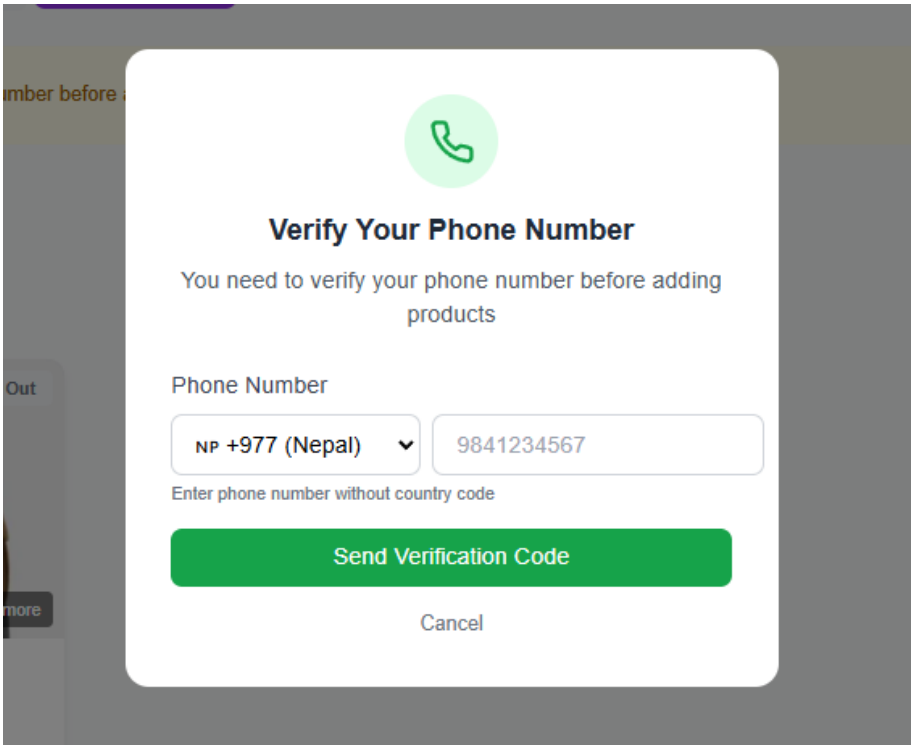


12. Admin Dashboard



ID	Name	Email	Role	Joined
2	Ram Thapa	ram@example.com	USER	12/10/2025
3	santos	santos@example.com	USER	12/10/2025
4	samir	samir@example.com	USER	12/11/2025
5	hello	hello@gmail.com	USER	12/11/2025
6	hello	hello1@gmail.com	USER	12/11/2025
7	hihello	hi@gmail.com	USER	12/11/2025
10	Santosh Shrestha	santos1234@gmail.com	USER	12/13/2025
14	Rusha Bade	rukishabade7@gmail.com	USER	4/5/2026
9	Santosh Shrestha	admin@gmail.com	ADMIN	12/11/2025
11	Samir	samir@gmail.com	USER	1/21/2026
12	Aryan	aryan@example.com	USER	3/9/2026
13	Seeshal Kayastha	seeshalkayastha@gmail.com	USER	4/3/2026
8	Santosh Shrestha	emperoropm@gmail.com	USER	12/11/2025

13. Products Approval Page



14. Phone Verification Page